

# Índice general

|     |                                                                        |    |
|-----|------------------------------------------------------------------------|----|
| I   | Introducción                                                           | 3  |
| 1.  | Instalación rápida (para quien ya conoce Python)                       | 5  |
| II  | Instalación paso a paso                                                | 6  |
| 2.  | Requisitos previos                                                     | 7  |
| 3.  | Qué es un entorno virtual y por qué usarlo                             | 8  |
| 4.  | Crear y activar un entorno virtual                                     | 9  |
| 5.  | Instalar <code>calccprop</code>                                        | 10 |
| 6.  | Nota para usuarios de Windows                                          | 11 |
| III | Variables proposicionales                                              | 12 |
| IV  | Conectivos lógicos                                                     | 14 |
| 7.  | Precedencia: pon paréntesis                                            | 16 |
| 8.  | No uses <code>and</code> , <code>or</code> ni <code>not</code>         | 17 |
| V   | La función <code>test</code>                                           | 18 |
| 9.  | Ejemplos básicos                                                       | 20 |
| 10. | <code>test</code> con fórmulas compuestas como hipótesis               | 21 |
| 11. | <code>True</code> y <code>False</code> como fórmulas                   | 22 |
| 12. | Las hipótesis van en una lista                                         | 23 |
| VI  | Funciones auxiliares                                                   | 24 |
| 13. | <code>unoDe</code> — exactamente uno verdadero                         | 25 |
| 14. | <code>alguno</code> — al menos uno verdadero                           | 26 |
| 15. | <code>noDerivable</code> — la afirmación no se deduce de las hipótesis | 27 |

|             |                                                                              |           |
|-------------|------------------------------------------------------------------------------|-----------|
| <b>VII</b>  | <b>Ver el razonamiento: <code>explica</code> y <code>tabla_verdad</code></b> | <b>28</b> |
| 16.         | <code>explica</code> — el razonamiento paso a paso                           | 30        |
| 17.         | <code>tabla_verdad</code> — todos los mundos posibles                        | 31        |
| <b>VIII</b> | <b>Funciones de bajo nivel</b>                                               | <b>32</b> |
| 18.         | <code>refuta</code> — un contraejemplo                                       | 34        |
| 19.         | <code>span</code> — evaluar una fórmula en un mundo concreto                 | 35        |
| 20.         | <code>extension</code> — el motor                                            | 36        |
| <b>IX</b>   | <b>Representación de las fórmulas</b>                                        | <b>37</b> |
| <b>X</b>    | <b>Referencia rápida</b>                                                     | <b>39</b> |

**Parte I**

**Introducción**

`calcprop` es una librería Python que implementa un motor de cálculo proposicional. Permite construir fórmulas lógicas combinando variables proposicionales con los conectivos habituales, y comprobar si una fórmula es consecuencia lógica de un conjunto de hipótesis.

Está diseñada para usarse como motor de razonamiento en aplicaciones que necesitan determinar automáticamente la veracidad de afirmaciones a partir de supuestos dados. El paquete [qbank](#) (publicado en PyPI como `calcprop-qbank`) lo utiliza para generar bancos de preguntas de opción múltiple: cada afirmación de una pregunta lleva asociada una fórmula, y `calcprop` decide si es verdadera, falsa o indecidible a partir de los supuestos del enunciado.

# Capítulo 1

## Instalación rápida (para quien ya conoce Python)

---

```
pip install calcprop
```

---

Si no estás familiarizado con Python o los entornos virtuales, lee la sección siguiente.

## Parte II

# Instalación paso a paso

## Capítulo 2

# Requisitos previos

`calccprop` requiere Python 3.9 o superior y no depende de ningún otro paquete. Comprueba si ya tienes Python instalado:

| Sistema | Comando en terminal            |
|---------|--------------------------------|
| Linux   | <code>python3 --version</code> |
| macOS   | <code>python3 --version</code> |
| Windows | <code>py --version</code>      |

Si el comando devuelve algo como `Python 3.11.2` estás listo. Si no, descarga Python desde [python.org](https://python.org) e instálalo (en Windows marca la opción *Add Python to PATH*).

## Capítulo 3

# Qué es un entorno virtual y por qué usarlo

Un entorno virtual es una carpeta que contiene una copia aislada de Python con sus propios paquetes instalados, independiente del resto del sistema. Esto evita conflictos entre proyectos que requieren versiones distintas de las mismas librerías.

## Capítulo 4

# Crear y activar un entorno virtual

Abre una terminal y sitúate en el directorio donde quieres trabajar:

```
mkdir mi_proyecto  
cd mi_proyecto
```

Crea el entorno virtual:

| Sistema       | Comando                            |
|---------------|------------------------------------|
| Linux / macOS | <code>python3 -m venv .venv</code> |
| Windows (cmd) | <code>py -m venv .venv</code>      |
| Windows (PS)  | <code>py -m venv .venv</code>      |

Actívalo:

| Sistema              | Comando                                 |
|----------------------|-----------------------------------------|
| Linux / macOS        | <code>source .venv/bin/activate</code>  |
| Windows (cmd)        | <code>.venv\Scripts\activate.bat</code> |
| Windows (PowerShell) | <code>.venv\Scripts\Activate.ps1</code> |

## Capítulo 5

# Instalar **calcprop**

Con el entorno activo:

---

```
pip install calcprop
```

---

Verifica la instalación:

---

```
python -c "from calcprop import *; print('calcprop OK!')"
```

---

## Capítulo 6

# Nota para usuarios de Windows

En Windows, dentro del entorno virtual, usa `python` (no `python3`). Si `py -m venv .venv` falla con un error de permisos en PowerShell, ejecuta antes:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

## Parte III

# Variables proposicionales

Las variables proposicionales son los átomos del cálculo proposicional. Se crean con la función `v()`, que acepta cualquier cadena de texto (o número) como nombre:

---

```
from calccprop import *
```

```
A = v("A")  
B = v("B")
```

---

Cada llamada a `v()` con el mismo argumento produce una variable igual a las anteriores, así que no hace falta guardarlas: `v(".^")` escrito en dos sitios distintos es la misma variable.

---

```
print(v("A") == v("A")) # True  
print(v("A") == v("B")) # False
```

---

True  
False

Parte IV

**Conectivos lógicos**

Las fórmulas se construyen combinando variables con los operadores de Python. Si **A** y **B** son fórmulas:

**-A** negación ( $\neg A$ ): *es falso que A*.

**A & B** conjunción ( $A \wedge B$ ): *A y B*.

**A | B** disyunción ( $A \vee B$ ): *A o B* (o ambas).

**A >> B** implicación ( $A \rightarrow B$ ): *si A entonces B*.

**A \*\* B** bicondicional ( $A \leftrightarrow B$ ): *A si y solo si B*.

Las fórmulas se pueden anidar libremente:

---

```
f1 = v("A") >> v("B")           # A implica B
f2 = -v("A") | v("B")           # no-A o B (equivalente a f1)
f3 = (v("A") & v("B")) ** v("C") # (A y B) si y solo si C
print(f1)
print(f2)
print(f3)
```

---

```
v('A') >> v('B')
-v('A') | v('B')
(v('A') & v('B')) ** v('C')
```

## Capítulo 7

# Precedencia: pon paréntesis

Los operadores son los de Python, y Python los evalúa con **su** orden de prioridad, que no es el habitual en lógica. De mayor a menor prioridad:

1. `**` (bicondicional),
2. `-` (negación),
3. `>>` (implicación),
4. `&` (conjunción),
5. `|` (disyunción).

Esto produce lecturas inesperadas:

- `A & B >> C` se lee como `A & (B >> C)`, y no como `(A & B) >> C`.
- `A | B >> C` se lee como `A | (B >> C)`.
- `-A ** B` se lee como `-(A ** B)`, y no como `(-A) ** B`.
- `A >> B >> C` se lee como `(A >> B) >> C`.

La regla práctica: **siempre que mezcles conectivos distintos, pon paréntesis**. Solo `-A & B` y `-A | B` (negación de una variable dentro de una conjunción o disyunción) son seguros sin ellos.

---

```
print( (v("A") & v("B")) >> v("C")) == (v("A") & (v("B") >> v("C"))) ) # True: ¡no es (A&B)>>C!
```

---

True

Si tienes dudas sobre cómo se ha leído una fórmula, imprímela: `calcprop` la muestra con paréntesis en cada operación anidada, de modo que la lectura sea inequívoca.

---

```
print( v("A") & v("B") >> v("C") )
```

---

```
v('A') & (v('B') >> v('C'))
```

## Capítulo 8

# No uses **and**, **or** ni **not**

Las palabras reservadas `and`, `or` y `not` de Python no se pueden redefinir, así que no sirven para construir fórmulas. Para evitar errores silenciosos, `calcprop` lanza un `TypeError` si se intenta:

---

```
try:
    v("A") and v("B")
except TypeError as e:
    print(e)
```

---

Una Proposición no tiene valor de verdad por sí misma: use `test(P, premisas)`. Para combinar proposiciones use los operadores

## Parte V

# La función **test**

`test(formula, hipotesis)` comprueba si una fórmula es consecuencia lógica de una lista de hipótesis. Devuelve `True` o `False`.

---

```
test(formula, hipotesis=[])
```

---

| Parámetro              | Descripción                                                                                                                    |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>formula</code>   | La fórmula cuya derivabilidad se comprueba. También acepta <code>True</code> o <code>False</code> , que se devuelven tal cual. |
| <code>hipotesis</code> | Lista de fórmulas que se asumen verdaderas. Por defecto vacía.                                                                 |

Que `test` devuelva `False` **no** significa que la fórmula sea falsa, sino que no se deduce de las hipótesis: puede ser falsa o puede ser indecidible con la información dada. Para distinguir ambos casos véase `noDerivable` más abajo.

## Capítulo 9

# Ejemplos básicos

---

```
# Modus ponens: de A y (A → B) se deriva B
print(test(v("B"), [v("A"), v("A") >> v("B")]))  # True

# Sin hipótesis, B no es derivable
print(test(v("B"), []))                          # False

# Una tautología siempre es verdadera
print(test(v("A") | ~v("A"), []))                 # True

# Una contradicción nunca es derivable
print(test(v("A") & ~v("A"), []))                 # False
```

---

True  
False  
True  
False

## Capítulo 10

# test con fórmulas compuestas como hipótesis

Las hipótesis pueden ser cualquier fórmula, no solo variables atómicas:

---

```
# De  $(A \rightarrow B)$  y  $(B \rightarrow C)$  se deriva  $(A \rightarrow C)$ 
print(test(v("A") >> v("C"),
          [v("A") >> v("B"), v("B") >> v("C")])) # True

# De  $(A \leftrightarrow B)$  se deriva tanto  $(A \rightarrow B)$  como  $(B \rightarrow A)$ 
print(test(v("A") >> v("B"), [v("A") ** v("B")])) # True
print(test(v("B") >> v("A"), [v("A") ** v("B")])) # True
```

---

True

True

True

## Capítulo 11

# True y False como fórmulas

Los literales Python `True` y `False` se pueden usar directamente como fórmula y como hipótesis. Como fórmula se devuelven tal cual (sin mirar las hipótesis); como hipótesis, `True` no aporta información y `False` hace derivable cualquier cosa:

---

```
print(test(True, []))      # True
print(test(False, []))    # False
print(test(v("A"), [True])) # False – True como hipótesis no aporta información
print(test(v("A"), [False])) # True – de una hipótesis falsa se deriva cualquier cosa
```

---

```
True
False
False
True
```

## Capítulo 12

# Las hipótesis van en una lista

Un despiste frecuente es pasar una sola hipótesis sin corchetes. `calcprop` lo detecta:

---

```
try:
    test(v("B"), v("A") >> v("B"))
except TypeError as e:
    print(e)
```

---

premisas debe ser una lista de proposiciones, por ejemplo `test(P, [v('A') >> v('B')])`.

**Parte VI**

**Funciones auxiliares**

## Capítulo 13

# unoDe — exactamente uno verdadero

`unoDe(A, B, C, ...)` construye una fórmula que es verdadera cuando exactamente uno de los argumentos es verdadero:

---

```
# Exactamente uno de A, B, C es verdadero
f = unoDe(v("A"), v("B"), v("C"))

# Si A es verdadero, B y C deben ser falsos
print(test(f, [v("A"), -v("B"), -v("C")])) # True
print(test(f, [v("A"), v("B")]))          # False (dos verdaderos)

# Y al revés: si exactamente uno es verdadero y es A, entonces B es falso
print(test(-v("B"), [f, v("A")]))          # True
```

---

True  
False  
True

Este es el uso típico en un banco de preguntas: el enunciado sortea uno de varios supuestos excluyentes (`unoDe`), y así la negación de los demás pasa a ser deducible.

## Capítulo 14

# alguno — al menos uno verdadero

`alguno(A, B, C, ...)` es equivalente a la disyunción  $A \mid B \mid C \mid \dots$ :

---

```
f = alguno(v("A"), v("B"), v("C"))
print(test(f, [v("B")])) # True
print(test(f, []))      # False
```

---

True  
False

## Capítulo 15

# noDerivable — la afirmación no se deduce de las hipótesis

`test(P, hipotesis)` responde a *¿se deduce P?*. A veces la pregunta interesante es la contraria: *¿es P indecible con lo que sabemos?* Eso es lo que expresa `noDerivable(P)`: es verdadera cuando P **no** es consecuencia lógica de las hipótesis.

---

```
H = [v("A") >> v("B"), v("A")]
print(test(noDerivable(v("B")), H))    # False: B sí se deduce
print(test(noDerivable(v("C")), H))    # True: C no se deduce
print(test(noDerivable(~v("C")), H))   # True: tampoco se deduce no-C
```

---

```
False
True
True
```

Obsérvese la diferencia entre las tres formas de preguntar por C:

`test(v("C"), H)` ¿es C verdadera (deducible)?

`test(~v("C"), H)` ¿es C falsa (deducible su negación)?

`test(noDerivable(v("C")), H)` ¿es C indecible?

Las dos primeras dan **False** en el ejemplo, y solo la tercera revela que C es indecible. En un banco de preguntas esto permite, por ejemplo, formular una cuestión solo cuando el enunciado *no* determina la respuesta (`precond = noDerivable(...)`).

`noDerivable` puede combinarse con los demás conectivos (`~noDerivable(P)` significa *P sí es deducible*), pero **no puede aparecer dentro de las hipótesis**: no es una afirmación sobre el mundo, sino sobre lo que las hipótesis permiten deducir. Si se intenta, `test` lanza un `ValueError`.

## Parte VII

**Ver el razonamiento: explica y  
tabla\_verdad**

Las dos funciones de esta sección no cambian ningún resultado: sirven para *entender* por qué `test` responde lo que responde. Ambas se imprimen como texto en la terminal y como una tabla o un árbol coloreado en un cuaderno Jupyter. El cuaderno `Introduccion.ipynb` las usa para presentar el cálculo proposicional desde cero.

## Capítulo 16

# explica — el razonamiento paso a paso

`explica(formula, hipotesis)` hace lo mismo que `test`, pero anotando cada paso. `calcprop` decide por el método de los *tableaux semánticos*: supone que existe un contraejemplo (hipótesis verdaderas y fórmula falsa), descompone esa suposición con reglas que se leen de las tablas de verdad de cada conectivo, y anota los valores de los átomos; cuando una rama exige dos valores distintos para el mismo átomo, se cierra. Si todas las ramas se cierran, no hay contraejemplo y la fórmula se deduce.

---

```
print(explica(v("B"), [v("A"), v("A") >> v("B")]))
```

---

¿Se deduce  $v('B')$  de las premisas  $v('A')$ ,  $v('A') \gg v('B')$  ?

Método: buscamos un contraejemplo, es decir, un mundo en el que las premisas sean V y la fórmula F. Si toda rama se cierra

Buscamos un mundo en el que  $v('B') = F$ ,  $v('A') = V$ ,  $v('A') \gg v('B') = V$ :

$v('B') = F \rightarrow$  anotamos  $v('B') = F$ .

$v('A') = V \rightarrow$  anotamos  $v('A') = V$ .

$v('A') \gg v('B') = V \rightarrow$  por la regla de  $\gg$ : o bien  $v('A') = F$ , o bien  $v('B') = V$

Rama 1 de 2: suponemos  $v('A') = F$

$v('A') = F \times$  contradice lo anotado ( $v('A') = V$ ): rama cerrada.

Rama 2 de 2: suponemos  $v('B') = V$

$v('B') = V \times$  contradice lo anotado ( $v('B') = F$ ): rama cerrada.

$\times$  Las 2 ramas se cierran.

Conclusión: todas las ramas se cierran, no hay contraejemplo.  $v('B')$  se deduce de las premisas: `test`  $\rightarrow$  `True`.

Cuando la fórmula no se deduce, la rama que queda abierta es el contraejemplo, y el objeto devuelto lo guarda en el atributo `contraejemplo` (el atributo `resultado` es el valor de `test`):

---

```
r = explica(v("A") >> v("C"), [v("A") >> v("B")])
print(r.resultado)
print(r.contraejemplo)
```

---

`False`

`{v('A'): True, v('C'): False, v('B'): True}`

## Capítulo 17

# tabla\_verdad — todos los mundos posibles

`tabla_verdad(formula, hipotesis[], subformulas=False)`= construye la tabla de verdad: una fila por cada combinación de valores de los átomos (V primero). Si se dan hipótesis, marca con \* las filas en que todas son verdaderas, que son las únicas que cuentan para la consecuencia lógica; la propiedad consecuencia coincide con `test`, pero calculada por fuerza bruta. Con `subformulas=True` se añaden las columnas intermedias, como en una tabla hecha a mano.

---

```
print(tabla_verdad(v("A"), [v("B"), v("A") >> v("B")]))
```

---

|   | v('A') |  | v('B') |  | v('A') >> v('B') |  | v('A') |
|---|--------|--|--------|--|------------------|--|--------|
|   |        |  |        |  |                  |  |        |
| * | V      |  | V      |  | V                |  | V      |
|   | V      |  | F      |  | F                |  | V      |
| * | F      |  | V      |  | V                |  | F      |
|   | F      |  | F      |  | V                |  | F      |

En la fila 3 las premisas son V pero `v('A')` es F: es un contraejemplo, `v('A')` no se deduce (`test → False`).

Sin hipótesis, las propiedades `tautologia` y `contradiccion` resumen la última columna:

---

```
print(tabla_verdad(v("A") | ~v("A")).tautologia)
print(tabla_verdad((v("A") >> v("B")) ** (~v("B") >> ~v("A"))), subformulas=True))
```

---

True

`v('A') | v('B') | v('A') >> v('B') | ~v('B') | ~v('A') | ~v('B') >> ~v('A') | (v('A') >> v('B')) ** (~v('B') >> ~v('A'))`

|   |  |   |  |   |  |   |  |   |  |   |  |   |
|---|--|---|--|---|--|---|--|---|--|---|--|---|
|   |  |   |  |   |  |   |  |   |  |   |  |   |
|   |  |   |  |   |  |   |  |   |  |   |  |   |
| V |  | V |  | V |  | F |  | F |  | V |  | V |
| V |  | F |  | F |  | V |  | F |  | F |  | V |
| F |  | V |  | V |  | F |  | V |  | V |  | V |
| F |  | F |  | V |  | V |  | V |  | V |  | V |

`(v('A') >> v('B')) ** (~v('B') >> ~v('A'))` es V en las 4 filas: es una tautología (`test → True`).

## Parte VIII

# Funciones de bajo nivel

Las funciones siguientes son las piezas con las que está construida `test`. En el uso normal no hacen falta, pero son útiles para depurar o para entender por qué una fórmula no se deduce.

## Capítulo 18

# refuta — un contraejemplo

`refuta(P, hipotesis)` busca una asignación de valores de verdad a las variables que hace ciertas las hipótesis y falsa  $P$ . Si la encuentra, la devuelve como diccionario; si no existe (es decir, si  $P$  se deduce), devuelve el diccionario vacío `{}`. `test(P, H)` es exactamente `not refuta(P, H)`.

---

```
print(refuta(v("A") >> v("C"), [v("A") >> v("B")]))
```

---

```
{v('A'): True, v('C'): False, v('B'): True}
```

El resultado se lee: *con  $A$  y  $B$  verdaderas y  $C$  falsa, la hipótesis se cumple pero  $A \rightarrow C$  no*. El contraejemplo no es único; `refuta` devuelve el primero que encuentra.

## Capítulo 19

# span — evaluar una fórmula en un mundo concreto

`span(F, P)` devuelve el valor de verdad de `P` cuando las variables toman los valores del diccionario `F`. Las variables que no aparecen en `F` se consideran falsas.

---

```
F = {v("A"): True, v("B"): False}
print(span(F, v("A") >> v("B"))) # False
print(span(F, v("A") | v("B")))  # True
```

---

```
False
True
```

## Capítulo 20

### **extension** — el motor

`extension(objetivo, valores, premisas)` es el algoritmo de *tableaux semánticos* que hay debajo de `refuta`: recibe una lista de pares (fórmula, valor deseado) y extiende el diccionario `valores` hasta satisfacerlos todos, o devuelve `{}` si es imposible. Está documentada en detalle en el código fuente literario `CalcProp.org`.

## Parte IX

# Representación de las fórmulas

Al imprimir una fórmula se obtiene código Python que la reconstruye. Es decir, `eval(repr(P))` vuelve a dar `P`. Cada operación anidada aparece entre paréntesis, así que la representación es a la vez inequívoca para Python y legible para una persona:

---

```
P = -(v("A") & v("B")) >> unoDe(v("C"), v("D"))
print(P)
print(eval(repr(P)) == P)
```

---

```
-(v('A') & v('B')) >> unoDe(v('C'), v('D'))
True
```

Esta propiedad es la que permite a `qbank` guardar las fórmulas como texto en ficheros JSON y recuperarlas después sin cambios.

Parte X

Referencia rápida

```

from calccprop import *

v("X")          → variable proposicional X
-v("X")          → negación de X  (no X)
v("X") & v("Y")  → conjunción    (X y Y)
v("X") | v("Y")  → disyunción    (X o Y)
v("X") >> v("Y") → implicación   (si X entonces Y)
v("X") ** v("Y") → bicondicional (X si y solo si Y)
True / False    → tautología / contradicción

Precedencia (de mayor a menor): **, -, >>, &, |
    → al mezclar conectivos, usa paréntesis: (A & B) >> C

test(formula, [h1, h2, ...]) → bool
    True si formula es consecuencia lógica de las hipótesis h1, h2, ...

unoDe(A, B, C, ...) → exactamente uno de los argumentos es verdadero
alguno(A, B, C, ...) → al menos uno de los argumentos es verdadero
noDerivable(P)      → P no se deduce de las hipótesis (P es indecible)
                    (solo en la fórmula, nunca en las hipótesis)

explica(formula, [h1, ...]) → razonamiento paso a paso (y .resultado, .contraejemplo)
tabla_verdad(formula, [h1, ...], subformulas=False) → tabla de verdad (y .consecuencia)

refuta(formula, [h1, ...]) → contraejemplo (dict) o {} si no lo hay
span({v("X"): True, ...}, formula) → valor de formula en ese mundo

```